THE DATA REFINERY — 21-DAY SPRINT (DAY 7)

Author: Prasad S Hiremath | **Date:** October 15, 2026 | **Topic:** Gaps & Islands, Pivoting & Atomic Upserts

1. Core Advanced SQL Patterns

1. The Gaps & Islands Problem: Identifies continuous ranges ('islands') vs. missing intervals ('gaps') using the parallel arithmetic trick: subtracting ROW_NUMBER() from ordered dates yields a constant group identifier.

2. Conditional Aggregation (Pivoting): Transforms row-level attributes into column-level summaries dynamically using SUM(CASE WHEN ... THEN 1 ELSE 0 END), ensuring high portability across database engines.

3. Atomic Upserts (MERGE): Synchronizes a target table with source streams in a single atomic transaction, updating matching records and inserting new ones without race conditions.

2. High-Frequency SQL Coding Challenges

Challenge 1: Gaps & Islands (Continuous Active Streak)
Write a query to find start/end dates of consecutive login streaks of 2+ days for each rider.

```
WITH numbered AS (SELECT rider_id, login_date, ROW_NUMBER() OVER (PARTITION BY rider_id ORDER BY login_date) as rn FROM rider_logins), islands AS (SELECT rider_id, login_date, DATE_SUB(login_date, INTERVAL rn DAY) as island_id FROM numbered) SELECT rider_id, MIN(login_date) AS start_date, MAX(login_date) AS end_date, COUNT(*) AS streak_days FROM islands GROUP BY rider_id, island_id HAVING COUNT(*) >= 2;
```

Challenge 2 (Bonus Code): Dynamic Pivoting (Monthly Sales Matrix)
Transform monthly revenue rows into distinct columns (Jan through Dec) using conditional aggregation.

```
SELECT store_id, SUM(CASE WHEN MONTH(sale_date) = 1 THEN revenue ELSE 0 END) AS jan_rev, SUM(CASE WHEN MONTH(sale_date) = 2 THEN revenue ELSE 0 END) AS feb_rev, SUM(revenue) AS total_annual_rev FROM sales GROUP BY store_id;
```

3. 10 Frequently Asked Interview MCQs & Solutions

Q1: Gaps & Islands Trick: Why does subtracting row number from an ordered date create a constant value for consecutive rows?

- **Answer:** B) Consecutive dates increment by 1 while ROW_NUMBER() increments by 1, making their difference static.
- **Reasoning:** Parallel arithmetic keeps date minus row number constant across unbroken sequences.

Q2: NOT IN vs NOT EXISTS: How does NOT IN behave compared to NOT EXISTS when the subquery returns a NULL value?

- **Answer:** A) NOT IN returns zero rows due to 3-valued logic, whereas NOT EXISTS handles NULL correctly.
- **Reasoning:** NULL propagation in NOT IN evaluates to UNKNOWN, filtering out all rows.

Q3: Atomic Upsert (MERGE): Why is MERGE preferred over separate DELETE and INSERT statements in data pipelines?

- **Answer:** B) MERGE executes matching, updates, and inserts atomically in a single pass, preventing race conditions.
- **Reasoning:** Atomicity prevents partial writes and lock contention in concurrent workloads.

Q4: Conditional Aggregation: What is the primary advantage of conditional aggregation over native PIVOT operators?

- **Answer:** A) It is universally supported across almost all SQL dialects with superior optimizer execution paths.
- **Reasoning:** Standard ANSI SQL compatibility avoids vendor-specific syntax locks.

Q5: Window Framing: What is the default ANSI SQL window frame when ORDER BY is specified without an explicit ROWS/RANGE?

- **Answer:** C) ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- **Reasoning:** Standard ANSI defaults to a cumulative running total frame when sorting.

Q6: Self-Join Mechanics: What is mandatory when performing a Self-Join on a single table?

- **Answer:** B) Aliasing the table into two distinct names (e.g., table AS a JOIN table AS b)
- **Reasoning:** Aliases distinguish table instances (e.g., employee vs manager).

Q7: Correlated Subqueries: Why are correlated subqueries generally discouraged in large-scale data warehouses?

- **Answer:** B) They execute row-by-row against the outer query, destroying vectorized MPP parallel processing.
- **Reasoning:** Row-by-row iteration bottlenecks distributed cluster execution.

Q8: Materialized Views: What is the primary operational overhead of Materialized Views?

- **Answer:** B) They require explicit refresh mechanisms to prevent stale data when base tables change.
- **Reasoning:** Persisted physical snapshots must be synchronized upon source mutations.

Q9: CTE Optimization: Do CTEs always improve query performance compared to inline subqueries?

- **Answer:** B) No, unless materialized by the query optimizer, standard non-recursive CTEs act primarily as inline macros.
- **Reasoning:** Optimizers inline CTEs unless specifically materialized or forced.

Q10: Hash Join vs Sort-Merge: When does a query engine typically choose a Hash Join over a Sort-Merge Join?

- **Answer:** A) When datasets are unsorted and fit into available memory for building in-memory hash tables.
- **Reasoning:** Hash joins are faster for unsorted inputs when memory permits hash table creation.